

Configuration Guide

This document explains all primary configuration paths for Zygos.

- Main config file: `config/default.yaml`
- Example provider config: `config/providers.example.yaml`
- Environment template: `.env.example`

Zygos uses **YAML config + environment variables**.

Configuration loading order

1. Internal defaults (from `src/config/loader.ts`)
 2. Values from your config file (default `config/default.yaml` , or custom via `--config`)
 3. Environment placeholders in YAML (e.g., `${OPENAI_API_KEY}`)
 4. Runtime CLI overrides (e.g., `--provider` , `--model` , `--rdt` , `--rdt-profile`)
-

Environment variables

Core

- `ZYGOS_DEBUG`
- `1` enables extra debug logging in provider/context components.

Provider credentials

- `OPENAI_API_KEY`
- `ANTHROPIC_API_KEY`

Database paths

- `ZYGOS_CONTEXT_DB` (default: `.zygos/context.db`)
 - `ZYGOS_LEARNING_DB` (default: `.zygos/learning.db`)
 - `ZYGOS_INTERVIEW_DB` (default: same as context DB)
-

Top-level config blocks

`config/default.yaml` includes these blocks:

- `runtime`
- `providers`
- `rdt`

- `learning`
- `interview`

runtime

Controls generic turn/tool runtime behavior.

- `maxTurns` (number): max model/tool cycles per turn.
- `maxToolCallsPerTurn` (number): hard cap on tool calls in one turn.
- `enableStreamingTools` (boolean): enables tool progress streaming.

Example:

```
runtime:
  maxTurns: 20
  maxToolCallsPerTurn: 8
  enableStreamingTools: false
```

providers

Controls routing, resilience, and provider credential settings.

providers.primary

Primary model route used first.

```
providers:
  primary:
    provider: ollama
    model: llama3.1:8b
    weight: 1
```

providers.fallbacks

Ordered fallback routes.

```
fallbacks:
- provider: openai
  model: gpt-4o-mini
  weight: 1
- provider: anthropic
  model: claude-3-5-haiku-latest
  weight: 1
```

providers.retry

Retry policy for transient failures.

- `maxAttempts`
- `baseDelayMs`
- `maxDelayMs`

- `jitterRatio`

providers.circuitBreaker

Protects system from repeatedly hitting unhealthy providers.

- `failureThreshold`
- `resetTimeoutMs`
- `halfOpenMaxRequests`

providers.rateLimit

Client-side request throttling.

- `maxRequestsPerMinute`
- `burst`

providers.observability

- `debug` (boolean): verbose route/provider diagnostics.

providers.gracefulDegradationMessage

Message returned when all routes fail.

providers.credentials

Per-provider transport/auth configuration.

Each provider block may include:

- `enabled` (boolean)
- `apiKey` (string, for API-key providers)
- `baseUrl` (string, for self-hosted/local providers)
- `timeoutMs` (number)
- `models` (string[]) model support hints
- `headers` (map) custom headers
- `weight` (number) route weighting metadata
- `requestSizeLimitBytes` (number) request size guardrail
- `requireApiKey` (boolean)

Provider setup recipes

OpenAI

```
providers:
  credentials:
    openai:
      enabled: true
      apiKey: ${OPENAI_API_KEY}
      requireApiKey: true
```

Get API key: <https://platform.openai.com/api-keys>

Anthropic

```
providers:
  credentials:
    anthropic:
      enabled: true
      apiKey: ${ANTHROPIC_API_KEY}
      requireApiKey: true
```

Get API key: <https://console.anthropic.com/>

Ollama

```
providers:
  credentials:
    ollama:
      enabled: true
      baseUrl: http://127.0.0.1:11434
      requireApiKey: false
```

Install/start Ollama: <https://ollama.com/>

vLLM (OpenAI-compatible)

```
providers:
  credentials:
    vllm:
      enabled: true
      baseUrl: http://127.0.0.1:8000/v1
      requireApiKey: false
```

vLLM docs: <https://docs.vllm.ai/>

rdt (Reasoning Depth Tuning)

Controls structured reasoning flow.

- `enabled` : globally enable RDT runtime
- `profile` : `shallow` | `balanced` | `deep`

rdt.prelude / recurrent / coda

Each stage supports:

- `enabled`
- `temperature`
- `maxTokens`
- `systemInstruction`

rdt.recurrent extras

- `minLoopIters`
- `maxLoopIters`
- `allowBacktracking`
- `allowParallelPaths`

rdt.loop

- `maxLoopIters`
- `minLoopIters`
- `maxRevisionDepth`

rdt.confidence

- `thresholds.earlyExit`
- `thresholds.revise`
- `thresholds.floor`
- `adaptive`
- `adaptUpDelta`
- `adaptDownDelta`
- `smoothingFactor`

rdt.attention

- `defaultMode`
- `switchByTask`
- `modeSwitchComplexityThreshold`
- `moe.enabled`
- `moe.routedExperts`
- `moe.sharedExperts`
- `moe.topK`
- `moe.maxParallelExperts`
- `moe.loadBalanceWindow`

rdt.quality

- `enableTraceLogging`
- `preserveReasoningChain`
- `computeAdaptive`
- `enableMultiHop`

learning

Controls self-learning proposal generation, A/B tests, and safe application policy.

- `enabled`
- `approvalMode` (`auto` / policy-based flow)
- `autoApplyLowRisk`
- `maxProposalsPerCycle`

- `minObservationsForProposal`
- `observeWindowSize`
- `maxModificationsPerHour`
- `maxToolCreationsPerDay`
- `abTestSampleSize`
- `maxLatencyRegressionRatio`
- `minSuccessRateGain`
- `maxResourceCostPerTestMs`

interview

Controls requirement interview gating and plan generation.

- `enabled`
- `requireForComplexBuilds`
- `complexityThreshold`
- `maxQuestions`
- `allowBypassForSimpleRequests`
- `allowOverrideByFlag`
- `template` (`auto` , `web_app` , `data_pipeline` , `api_service` , `tool_utility` , `general`)

Feature toggles quick reference

- RDT: `rdt.enabled`
- Learning system: `learning.enabled`
- Interviewer: `interview.enabled`

You can also force runtime behavior from CLI:

```
npm run dev -- "<msg>" --rdt true --rdt-profile deep
npm run dev -- "<msg>" --provider ollama --model llama3.1:8b
```

Configuration profiles

Dev profile (fast local)

```
providers:
  primary: { provider: custom, model: demo, weight: 1 }
  fallbacks: []
rdt:
  enabled: false
learning:
  enabled: true
interview:
  enabled: true
```

Prod profile (resilient multi-provider)

```
providers:
  primary: { provider: openai, model: gpt-4o-mini, weight: 1 }
  fallbacks:
    - { provider: anthropic, model: claude-3-5-haiku-latest, weight: 1 }
  retry: { maxAttempts: 3, baseDelayMs: 300, maxDelayMs: 5000, jitterRatio: 0.2 }
  circuitBreaker: { failureThreshold: 3, resetTimeoutMs: 30000, halfOpenMaxRequests:
1 }
rdt:
  enabled: true
  profile: balanced
```

Custom profile file

Create `config/my-profile.yaml` and run:

```
npm run dev -- "<message>" --config config/my-profile.yaml
```

Database configuration

Zygos uses SQLite for persistence.

- Context/history DB: `ZYGOS_CONTEXT_DB` (default `.zygos/context.db`)
- Learning DB: `ZYGOS_LEARNING_DB` (default `.zygos/learning.db`)
- Interview DB: `ZYGOS_INTERVIEW_DB` (default uses context DB)

Recommended:

- Use project-local paths in development
 - Use mounted persistent volume paths in production
 - Back up DB files if interview plans/history are important
-

Security best practices

1. **Do not commit** `.env` files with real keys.
 2. Keep API keys in secure secret managers in production.
 3. Set provider `requestSizeLimitBytes` defensively.
 4. Prefer HTTPS provider endpoints for remote providers.
 5. Limit debug logging (`ZYGOS_DEBUG=0` in production).
 6. Restrict filesystem permissions for SQLite DB paths.
 7. Rotate provider keys regularly.
 8. Validate custom tool behavior before enabling auto-learning in sensitive environments.
-

Validation

Run:

```
npm run verify
```

This verifies installation prerequisites and confirms Zygus can initialize with your current configuration.